

BandDPER

Band-Aware Dual-Perspective Evaluation ResNet

Mathieu WAHARTE

April 2026

Contents

1	Analysis	3
1.1	Problem Statement	3
1.2	Requirements	3
1.3	Why a Neural Network	3
1.4	Existing Models	4
2	Model Architecture	5
2.1	Layers	6
2.1.1	Input	6
2.1.2	Shared Spatial Encoder	8
2.1.3	Residual Trunk	10
2.1.4	Output Head	13
3	Training	14
3.1	Data Generation (Minimax Bootstrapping)	14
3.2	Loss Function	14
3.3	Optimizer and Training Hyperparameters	14
3.4	Training Hardware	14
3.5	Export Pipeline	15
4	Inference	16
5	Optimizations	17
5.1	Masks, maps and channels	17
5.2	Iterative re-bootstrap rounds	17
5.3	Weighted value loss	17

5.4	SE blocks	17
5.5	Depthwise separable convolutions	17
5.6	Policy head for move ordering	17
5.7	Quantization	18
5.8	Distillation	19
5.9	Rejected optimizations	19
6	References	20
6.1	Papers and architecture references	20
6.2	Tools and implementation references	20

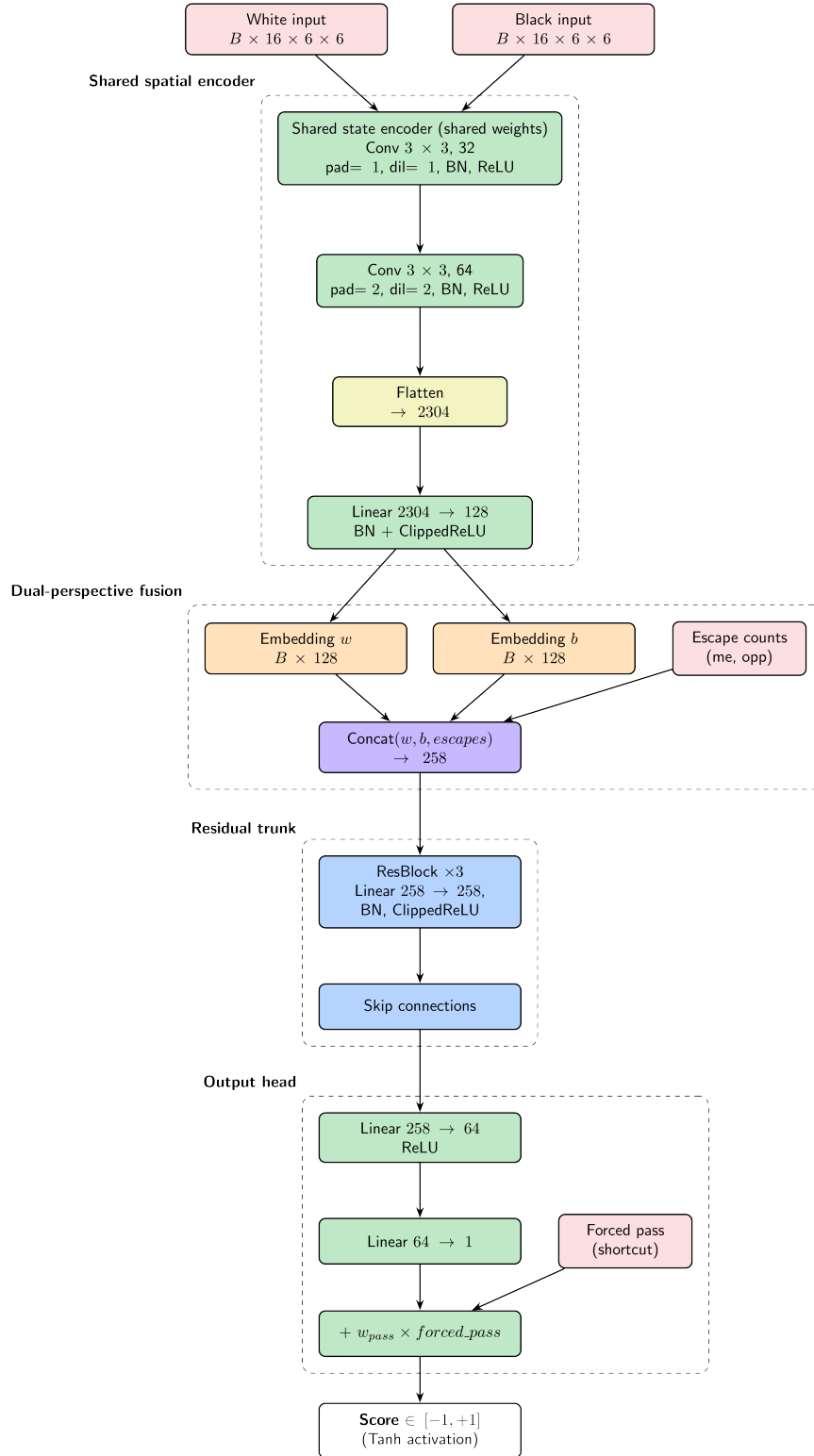


Figure 1: Model architecture flowchart for BandDPER.

1 Analysis

1.1 Problem Statement

Since the game is a full-information, deterministic, zero-sum game with a small board (6×6) and a fixed but non-uniform topology (the band map), strong domain knowledge can be leveraged to design an efficient evaluation function. The key challenge is to capture the complex interactions between pieces, the band constraints (which create strong local dependencies between adjacent squares), and the strategic patterns that emerge from the unique movement rules.

1.2 Requirements

The evaluation function returns a scalar in $[-1, +1]$, where -1 represents a losing position for the current player, $+1$ represents a winning position, and intermediate values represent varying degrees of advantage.

The function must capture several key signals:

- Which pieces can legally move right now (band constraint)
- How many moves each side has (mobility asymmetry)
- Proximity of attackers to the opponent's unicorn, weighted by band alignment and measured in a unicorn-relative coordinate frame
- Unicorn escape routes (counted explicitly as a scalar signal)
- Structural patterns (corridor control, band-type distribution)
- Tempo forcing: which band the current move lands on determines the opponent's forced departure
- Forced-pass detection: whether the current move leaves the opponent with no legal response

1.3 Why a Neural Network

Compared to our handcrafted evaluation function which already captures the most obvious signals, a neural network can learn nonlinear interactions, generalize easily from training data and capture structural patterns that were not encoded in our formula. The main drawback is that it would be slower but according to [AlphaZero](#), a better evaluation with a shallower depth can outperform a deeper search with a worse evaluation. Furthermore, the game is very small (6×6 with 12 pieces) and networks are heavily researched and inference optimizations can make it nearly as fast as a handcrafted eval.

1.4 Existing Models

Model	Notes	Recommendation
MLP	Fast	Avoid, no spatial awareness or relational reasoning
CNN	Exploits local spatial patterns; fast enough	OK but translation equivariance may be wrong for the non-uniform band map
NNUE	Fast incremental updates, dual-perspective design, ClippedReLU $[0, 1]$ activation	Doesn't fit this problem size, but dual-perspective design, ClippedReLU, king-relative features, and direct output shortcuts are worth borrowing
ResNet	Good when a baseline evaluation exists	Good, but lacks inherent spatial awareness
GNN	Movement graph; invariant to symmetries	Avoid, dynamic graph changes, implementation complexity, and slower performance

Table 1: Existing model candidates and recommendations.

2 Model Architecture

This architecture blends known components into a novel evaluation network:

- **Siamese CNN spatial encoder** with shared weights and dual-perspective input
- **Band topology and game signals encoded as fixed and dynamic input channels** (16 channels total)
- **Unicorn-relative attacker map** inspired by NNUE’s HalfKP king-relative encoding
- **Residual trunk** with ClippedReLU at the perspective boundary
- **Scalar injection** of escape counts at the trunk boundary
- **Direct output shortcut** for forced-pass signal, bypassing the trunk

Note on the name: “Dual-Perspective” refers to player-perspective symmetry enforced via shared Siamese weights. The architecture deliberately breaks spatial translation equivariance in two places: fixed band-map channels (4–6) encode absolute square identity, and channel 15 recentres around the unicorn.

Core design elements:

- **Siamese Network** (Shared-weight dual-perspective encoding, [Bromley et al. - 1993](#))
- **ResNet** (CNN + residual blocks, [He et al. - 2015](#))
- **Atrous convolution** (Dilated convolutions for range, [Chen et al. - 2015](#))
- **ClippedReLU** bounded activations ([NNUE - 2018](#))
- **Dual accumulator** for game positions ([NNUE - 2018](#))
- **Unicorn-relative encoding** inspired by HalfKP king-relative features ([NNUE - 2018](#))
- **Direct output shortcut** for near-linear signals ([Stockfish HalfKAv2 PSQT bypass - 2021](#))
- **Value head** (Value network for board games, [AlphaZero - 2017](#))

Parameter Count

Table 2: Approximate parameter count by component.

Component	Parameters
Encoder (shared, used $\times 2$)	$\sim 312,000$
ResBlock $\times 3$ (dim=258)	$\sim 402,648$
Output head	$\sim 16,577$
Total	$\sim 730,625$

This model would be around 2.9 MB in float32. The reduced variant with `embed_dim=64`, `num_res_blocks=2` has around **185K parameters**.

2.1 Layers

2.1.1 Input

For each perspective we have a $[16, 6, 6]$ tensor. For each board position, we have 16 channels:

Table 3: Input channels for each perspective tensor.

Channel	Content	Type
0	My paladin positions	Dynamic 0/1
1	My unicorn position	Dynamic 0/1
2	Opponent paladin positions	Dynamic 0/1
3	Opponent unicorn position	Dynamic 0/1
4	Band-1 square mask	Fixed 0/1
5	Band-2 square mask	Fixed 0/1
6	Band-3 square mask	Fixed 0/1
7	Departure constraint mask	Dynamic 0/1
8	Band-1 legal landing squares	Dynamic 0/1
9	Band-2 legal landing squares	Dynamic 0/1
10	Band-3 legal landing squares	Dynamic 0/1
11	My row occupancy fraction	Dynamic $[0, 1]$
12	Opponent row occupancy fraction	Dynamic $[0, 1]$
13	My column occupancy fraction	Dynamic $[0, 1]$
14	Opponent column occupancy fraction	Dynamic $[0, 1]$
15	Unicorn-relative opponent attacker map	Dynamic 0/1

Band masks (ch. 4–6) are fixed binary inputs representing the board’s spatial structure.

Channels 8–10 makes the tempo-forcing mechanic explicit.

Channel 15 places opponent paladins in a coordinate frame centered on our unicorn (anchor at row=2, col=2), inspired by HalfKP’s king-relative encoding. This lets the CNN learn threat patterns that are translation-invariant relative to the unicorn.

This encodes the full current board state. The CNN will learn the spatial correlations between pieces and band types. Having two perspectives allows us to follow the **Siamese network principle**, where the same encoder processes both perspectives with shared weights, enabling the network to learn a unified representation of the game state from both players’ viewpoints. Indeed, a position that is good for one player is bad for the other (menacing the unicorn is universally bad), so the network can learn to extract features that are relevant for both sides without needing separate encoders.

In addition, **3 scalars are injected** outside the CNN:

Table 4: Scalar injections.

Scalar	Content	Injection point
<code>escape_me</code>	Legal move count for my unicorn (0–16)	Concatenated into <code>h</code>
<code>escape_opp</code>	Legal move count for opponent unicorn	Concatenated into <code>h</code>
<code>forced_pass</code>	1 if opponent has no legal reply	Direct output bypass

The escape counts are appended to `h` after Siamese fusion. The forced-pass bit bypasses the trunk entirely via a single learned scalar weight `w_pass` added directly to the output before `tanh`. This is intentional: `tanh` bounds the sum correctly, and since `w_pass` is initialised to zero

and trained from data, it learns an appropriate magnitude without requiring additional scaling.

2.1.2 Shared Spatial Encoder

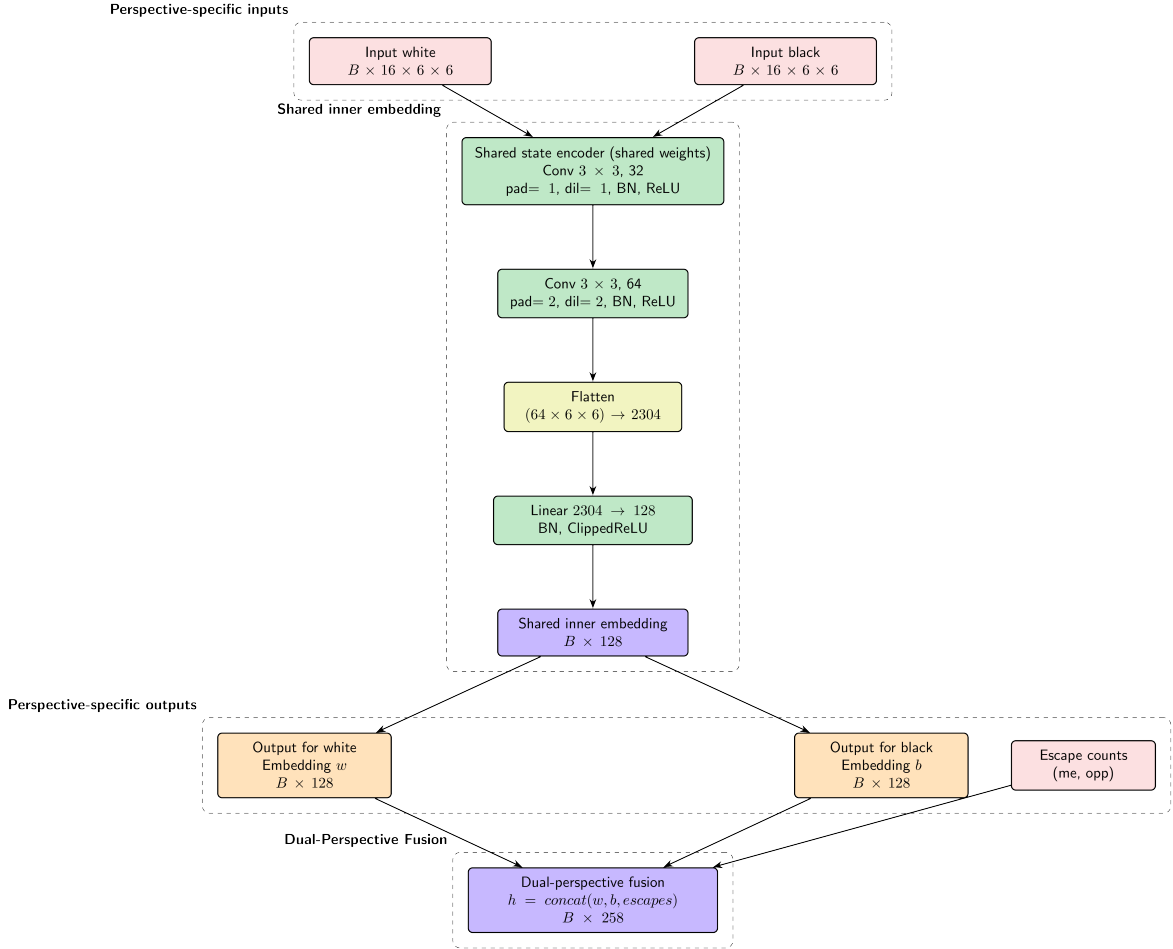


Figure 2: Shared Spatial Encoder used for both player perspectives (shared weights).

Figure 2 details the encoder stack used to extract perspective-specific embeddings. With B the batch size.

Notes:

- The first convolution captures **local spatial patterns** and piece interactions, which are crucial for evaluating immediate threats and opportunities. Its receptive field of 3×3 allows it to see adjacent pieces and band types. Importantly, broken-path movement of band=2 pieces reaches exactly the diagonal neighbours captured by this 3×3 kernel - so this convolution is now the primary movement-range detector.
- The second convolution with dilation=2 **expands the receptive field** (effective 5×5) to capture medium-range interactions such as L shaped movement patterns and corridor control.
- **BatchNorm**: Applied after each convolution to stabilize training and accelerate convergence.
- **ClippedReLU**: Used in the encoder output to ensure all values are in the range $[0, 1]$, preventing one perspective from dominating the other due to scale differences.

- **Dual-Perspective Fusion:** The outputs from both perspectives are concatenated to form a unified representation $\mathbf{h}$ of shape $[B, 256]$, then escape counts are appended giving $\mathbf{h}$ shape $[B, 258]$. Always pass inputs in the order `[current_player, opponent]`.
- Both perspectives are processed through the same encoder (shared weights) to learn a unified representation of the game state, enabling the network to generalize patterns that are relevant for both players (Siamese network principle).

2.1.3 Residual Trunk

There is a trunk of 3 ResBlocks with skip connections and dropout regularization. Each ResBlock consists of:

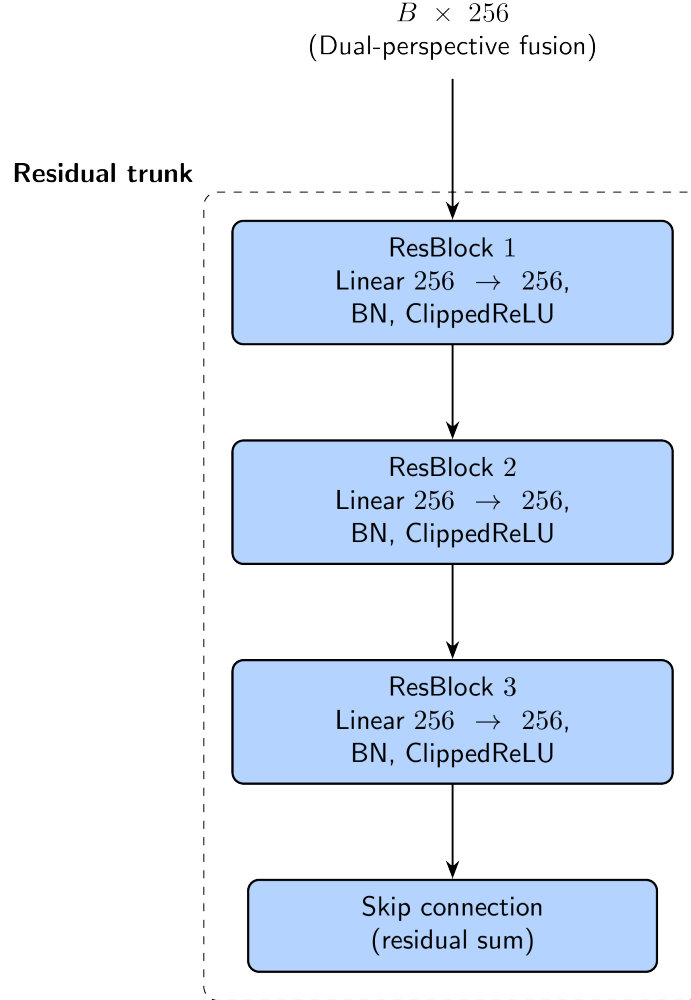


Figure 3: Residual trunk structure with stacked ResBlocks and skip connections.

Figure 3 shows the repeated residual pattern used to refine the fused representation.

A residual block computes an output y as the sum of its input x and a learned function $F(x)$: $y = F(x) + x$. The function F is typically a small neural network (e.g., two linear layers with an activation in between). This structure allows the block to learn a residual function (essentially a correction to its input) rather than needing to learn a full transformation from scratch. Additionally, the skip connection (the “ $+x$ ” part) allows gradients to flow directly through the block during backpropagation, which helps mitigate the vanishing gradient problem and enables training of deeper networks. This also provides graceful degradation as, if a block learns to output zero (i.e., $F(x) \approx 0$), it effectively becomes an identity function, allowing the network to skip it without harming performance.

Input dimension: 258 (256 Siamese CNN output + 2 escape scalars).

The current PyTorch implementation applies `Dropout(0.2)` after each linear layer in every

ResBlock and before the first linear layer in the output head. This regularizes the bootstrapped self-play training setup without changing the inference-time graph structure.

Notes:

- **Why residual blocks:** The network learns *corrections* on top of a near-correct base representation. This is the right structure when training via minimax bootstrapping. Skip connections prevent vanishing gradients with depth.
- **Why 3 ResBlocks:** Maps to Escampe’s three natural abstraction levels (proximity/unicorn-relative < band interaction/tempo < forced-pass/escape dynamics). Matches empirical optimum from board game literature at this parameter scale ([AlphaZero](#), [Train on Small, Play the Large: scaling up board games with AlphaZero and GNN](#), [Mastering Chess with a Transformer Model](#)). A smaller block count wouldn’t capture the full complexity of this game and going to three blocks is worth the additional cost. Also, with ClippedReLU, more than 3 blocks would lead to training instability as the gradient path through the skip connections becomes dominant and the $F(x)$ branch receives progressively weaker gradients. Meaning wise, after 3 levels of composition, further blocks learn redundant representations since the positional complexity is fundamentally bounded by the board size and piece interactions.
- **Why dropout in ResBlocks:** The model is trained on bootstrapped self-play labels, so dropout helps reduce overfitting without removing the residual signal or changing the inference-time architecture.
- **Why ClippedReLU in ResBlocks:** Keeps activations bounded at every depth, preventing value explosion through skip connections.
- **Scalar injection at trunk input:** Escape counts are global signals that do not have a natural spatial representation. Injecting them at $\mathbf{h}$ (after the CNN, before the trunk) lets the ResBlocks learn to combine spatial features with game-state scalars. The trunk’s Linear($258 \rightarrow 258$) layer learns the projection automatically.

2.1.4 Output Head

The output head uses a direct bypass for forced pass signals, inspired by [Stockfish’s PSQT direct-to-output shortcut](#):

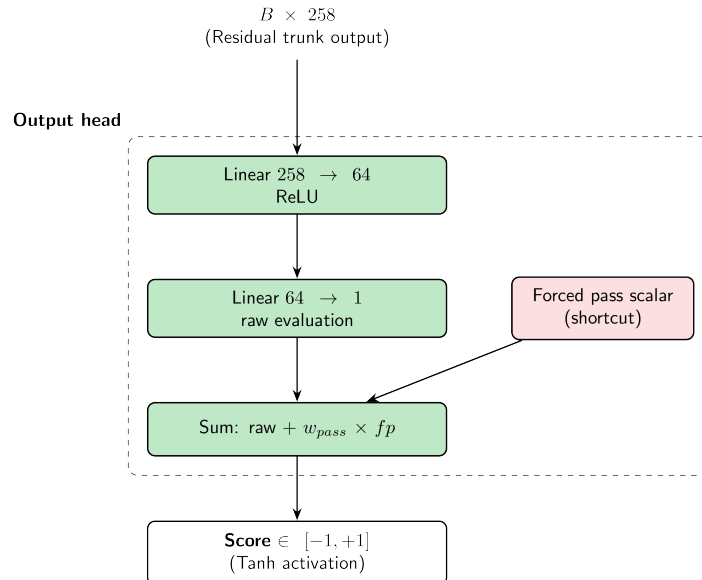


Figure 4: Output head producing a bounded scalar evaluation.

w_{pass} is a single learned scalar parameter. Wiring the forced pass signal directly to the output bypasses the trunk entirely, giving this near-linear signal maximum gradient and preventing the trunk from having to learn to route it. The output head also uses dropout before its first linear layer, matching the regularization used in the trunk.

This mirrors Stockfish HalfKAv2’s **FullThreats** feature set, which is passed directly to the output layer for a similar reason - some signals have a nearly linear effect on evaluation, and the hidden layers add noise rather than value.

No ClippedReLU on the output since Tanh already bounds the final value. The output is a scalar representing the evaluation from the current player’s perspective.

3 Training

3.1 Data Generation (Minimax Bootstrapping)

The existing alpha-beta engine is used at depth 5–7 to label positions with ground-truth scores. The network learns to approximate these deep search evaluations. Terminal positions (unicorn captured) are labelled exactly 1.0. All other positions are normalised as `score_label = minimax(board, depth=5) / MATE_SCORE`, where `MATE_SCORE` is the score returned by the engine for an immediate win (a large constant, e.g. 100_000). This normalises scores to $[-1, 1]$.

The data generation procedure:

1. Reset the board to a random legal opening configuration.
2. Play a mixed game: each ply either selects a random legal move (probability 0.7) or the engine’s best move at depth 2 (probability 0.3), to generate diverse positions.
3. At each ply after the 4th, label the current position with probability 0.3 using the depth-5 engine.
4. Continue until the game ends or 100 plies are reached.

Approximately **50K–100K labeled positions** are sufficient for the network to learn meaningful patterns without overfitting. Training examples store (`board_state`, `last_landing_band`, `label`) triples, not just (`board_state`, `label`), because both the departure constraint mask and the forced-pass signal require the last landing band. Forced-pass positions are rare ($\sim 2\text{--}5\%$ of positions); the direct shortcut ensures they receive strong gradient despite low frequency. Iterative re-bootstrap rounds at depth 6–8 using the trained network as evaluation function can further improve label quality.

3.2 Loss Function

We use **Mean Squared Error** (MSE) loss between the network’s predicted score and the minimax score label:

```
loss = F.mse_loss(prediction, target)
```

3.3 Optimizer and Training Hyperparameters

We use **AdamW** (`lr=1e-3`, `weight_decay=1e-4`) with **CosineAnnealingWarmRestarts** to escape plateaus during iterative bootstrapping rounds. We also apply **gradient clipping** (`clip_grad_norm=1.0`) to prevent exploding gradients.

3.4 Training Hardware

Training was performed on the free Google Colab Tier (4h T4 GPU), which is sufficient for this model size and dataset.

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118
```

3.5 Export Pipeline

```
# 1. Fold BatchNorm into conv/linear weights
def fold_batchnorm_conv(conv, bn):
    std = (bn.running_var + bn.eps).sqrt()
    /* Lines 216-218 omitted */
    return w_new, b_new

# 2. Export all weights to JSON
weights = {k: v.tolist() for k, v in model.state_dict().items()}
weights["band1_mask"] = BAND_MASK[1]
weights["band2_mask"] = BAND_MASK[2]
weights["band3_mask"] = BAND_MASK[3]
weights["w_forced_pass"] = model.w_forced_pass.item()
json.dump(weights, open("escampe_net_weights.json", "w"))

# 3. Save .pth for resuming training
torch.save(model.state_dict(), "escampe_net.pth")

# 4. Push to HuggingFace
api.upload_file("escampe_net.pth",
                path_in_repo="escampe_net.pth",
                repo_id="mathieu-waharte/escampe-eval")
```

4 Inference

Weights are loaded once at startup; the forward pass is then executed manually in Java with pre-allocated buffers to eliminate GC pressure in the hot path:

```
// Pre-allocate all activation buffers (no GC pressure in hot path)
float[][][] convBuf1, convBuf2;
float[] encBuf, wEmbed, bEmbed, trunk, h1, outBuf;
float wForcedPass; // w_pass: loaded once at startup

public float evaluate(EscapeBoard board, int lastLandingBand) {
    float[][][] xMe = boardToTensor(board, board.currentPlayer(), lastLandingBand);
    float[][][] xOpp = boardToTensor(board, board.opponent(), lastLandingBand);

    float[] w = encode(xMe); // [128], ClippedReLU output
    float[] b = encode(xOpp); // [128]

    // Scalar signals: escape counts, normalized to [0,1]
    float escapeMe = computeUnicornEscapeCount(board, board.currentPlayer()) / 16.0f;
    float escapeOpp = computeUnicornEscapeCount(board, board.opponent()) / 16.0f;

    // Concatenate perspectives + escape scalars [258]
    float[] h = concat(w, b, new float[]{escapeMe, escapeOpp});

    // ResBlocks
    for (ResBlockWeights rb : resBlocks) {
        h = resBlock(h, rb);
    }

    // Output head
    float[] out = linear(h, wOut1, bOut1); // [64]
    out = relu(out);
    float raw = linear(out, wOut2, bOut2)[0]; // [1]

    // Forced-pass: direct output shortcut, bypasses trunk
    int forcedPass = computeForcedPass(board, lastLandingBand);
    raw += wForcedPass * forcedPass;

    return (float) Math.tanh(raw);
}
```


5 Optimizations

5.1 Masks, maps and channels

Band masks, departure masks, band-landing maps, row/column occupancy maps, unicorn-relative attacker maps, escape scalars, and the forced-pass direct shortcut provide the strongest low-cost inductive bias improvements for this setting. They give the network explicit information about the board’s spatial structure, piece interactions, and game-specific mechanics that are critical for effective learning. Currently active inputs:

- Row/column occupancy maps (channels 11–14)
- Band-landing maps (channels 8–10)
- Unicorn escape counts (scalars at **h**)
- Forced-pass direct shortcut (scalar at output)
- Unicorn-relative attacker map (channel 15)

5.2 Iterative re-bootstrap rounds

After training the initial network, we can use it to generate new labels by running a deeper minimax search (depth 6–8) with the network as its evaluation function. This iterative bootstrapping is a key technique used in AlphaZero.

5.3 Weighted value loss

We can modify the MSE loss to give more weight to positions that are closer to winning or losing.

5.4 SE blocks

Squeeze-and-Excitation blocks can be added after convolutional layers to allow the network to learn channel-wise attention. Lightweight and easily integrated.

5.5 Depthwise separable convolutions

Replacing standard convolutions in the encoder with depthwise separable convolutions can reduce parameters and computational cost while maintaining performance.

5.6 Policy head for move ordering

We propose adding a **policy head** that outputs a probability distribution over the ~ 96 possible moves (due to broken-path movement reaching 4/8/16 unique destinations for band 1/2/3 respectively from interior squares). This head is used purely for move ordering in alpha-beta search.

```

policy_head = nn.Sequential(
    Linear(258, 64),
    ReLU(),
    Linear(64, 96),
    Softmax(dim=-1)
)

```

Then for each node in the search tree:

1. Generate legal moves using the existing move generator (very fast)
2. Call the policy head to obtain move probabilities; sort moves by descending probability.
3. Run alpha-beta with well-ordered moves for stronger pruning
4. Use handcrafted eval at leaf nodes `/!\` unless network inference is fast enough

5.7 Quantization

Table 5: Quantization choices by layer.

Layer	Params	Quantize?	Why
Conv layers (encoder)	~33K	INT8	Weights well-distributed; ClippedReLU bounds activations
Linear proj (1152 $\rightarrow$ 128)	~148K	INT8	Largest layer; most to gain
ResBlock linears	~402K	INT8	Clean bounded range after ClippedReLU
Output Linear(64 $\rightarrow$ 1)	65	keep float32	Tiny; output precision matters
<code>w_forced_pass</code>	1	keep float32	Single scalar; precision matters
Band mask channels	fixed	N/A	Already binary (0/1)

In ONNX/PyTorch:

```

model.eval()
model_q = torch.quantization.quantize_dynamic(
    model,
    {nn.Linear, nn.Conv2d},
    dtype=torch.qint8
)
torch.onnx.export(model_q, (x_w, x_b, esc_me, esc_opp, forced_pass),
    "escampe_net_q8.onnx")

```

And in Java with ONNX Runtime:

```

// pom.xml: com.microsoft.onnxruntime:onnxruntime:1.17.0
OrtEnvironment env = OrtEnvironment.getEnvironment();
OrtSession session = env.createSession("escampe_net_q8.onnx");
// Target: ~2--5\, \textmu s inference per position on modern hardware (to be
    benchmarked)

```

To evaluate the quantization performance, we propose to compare the score predictions of the quantized model against the original float32 model on a large set of random board positions. We then compute the mean absolute error (MAE) and mean squared error (MSE) between the two sets of predictions with 95, 99, 99.99 confidence intervals (to check if edge cases don't get degraded). Additionally, we can run a small tournament of 1000 self-play games using both models as evaluation functions in the search, and compare their win rates to see if the quantization has a significant impact on playing strength.

5.8 Distillation

Train a smaller “student” network to mimic the outputs of the larger “teacher” network using KL divergence + MSE distillation loss.

[Rapfi: Distilling Efficient Neural Network for the Game of Gomoku](#)

5.9 Rejected optimizations

- **$1 \times 6 / 6 \times 1$ asymmetric kernels:** Were originally considered to capture row/column patterns due to a misunderstanding of the movement rules. However, with broken-path movement, pieces can reach both orthogonal and diagonal neighbours, so a standard 3×3 kernel is more appropriate to capture all local interactions.
- **Attention pooling / self-attention in encoder:** Conflicts with incremental-style updates, global interaction cost excessive for board size.
- **NNUE-style incremental updates:** Architecture is convolutional and not naturally NNUE-like.
- **LayerNorm replacing folded BatchNorm:** Export pipeline folds BatchNorm away at inference.
- **Pre-activation ResNet:** Too little upside for only 3 residual blocks.
- **DenseNet-style dense connections:** Memory overhead without obvious benefit.
- **Full transformerization:** Too expensive and poorly matched for latency target and board size.

6 References

6.1 Papers and architecture references

- [Bromley et al. - 1993](#): Siamese network / shared-weight dual-perspective encoding
- [He et al. - 2015](#): ResNet residual blocks
- [Chen et al. - 2015](#): Atrous (dilated) convolutions
- [AlphaZero - 2017](#): Value head and improved evaluation via search
- [Train on Small, Play the Large: scaling up board games with AlphaZero and GNN](#): Small-board AlphaZero scaling
- [Mastering Chess with a Transformer Model](#): Transformer-based chess scaling insights
- [NNUE - 2018](#): ClippedReLU, dual accumulator, HalfKP king-relative features, direct output shortcuts
- [Stockfish HalfKAv2 architecture](#): FullThreats direct-to-output connection (source of J shortcut idea)
- [Stockfish NNUE - Chessprogramming wiki](#): HalfKP/HalfKAv2 feature set design rationale
- [Rapfi: Distilling Efficient Neural Network for the Game of Gomoku](#): Distillation techniques for board game evaluation networks

6.2 Tools and implementation references

- [PyTorch CUDA 11.8 wheels](#): GPU build used for training on GTX 1080
- [ONNX Runtime](#): Inference engine referenced for quantized model execution